Kas_SwinV2_Tiny_Kod_Calisma_Mantigi_Teknik_Raporu.pdf - local repository text copy

Eyebrow ROI Swin V2 Tiny - Kod Çalışma Mantığı ve Teknik Açıklama Raporu
Sayfa 1

EYEBROW ROI SWIN V2 TINY
Kod Çalışma Mantığı ve Teknik Açıklama Raporu

Kaş bölgesi üzerinden deepfake tespiti için Swin V2 Tiny tabanlı eğitim, doğrulama, test ve kalite kontrol pipeline'ı

İncelenen notebook: Eyebrow_SwinV2_Tiny_PDF_Standard_Nazlican(1).ipynb

Hazırlanma tarihi: 7 Ağustos 2026

Eyebrow ROI Swin V2 Tiny - Kod Çalışma Mantığı ve Teknik Açıklama Raporu
Sayfa 2

1. Raporun Amacı

Bu rapor, verilen Colab/Jupyter notebook'unun ne yaptığını, hangi veriyi kullandığını, veriyi nasıl doğruladığını, kaş ROI görüntülerini Swin V2 Tiny modeline nasıl hazırladığını, modelin nasıl eğitildiğini, checkpoint ve resume mekanizmasının nasıl çalıştığını ve deney sonunda hangi çıktıların üretildiğini teknik fakat okunabilir bir dille açıklamak amacıyla hazırlanmıştır.

Notebook'un temel görevi, daha önce kırpılmış kaş bölgesi görüntülerinden gerçek (real) ve sahte/deepfake (fake) sınıflarını ayırabilen ikili bir sınıflandırıcı eğitmektir. Kod sadece model eğitmekle kalmaz; veri bütünlüğü, train/validation/test ayrımı, kaynak video sızıntısı, sayısal kararlılık, checkpoint sürekliliği ve çıktı dosyalarının eksiksizliği için bir dizi kalite kapısı da uygular.

2. Kısaca: Kod Ne Yapıyor?

1. local source package'ı bağlar ve AISC DeepFake Çalışmaları içindeki kaş ROI veri klasörünü bulur.
2. Kaş ROI metadata dosyasını ve kaynak videoyu doğrulamak için frame seçim metadata dosyasını okur.
3. Yalnızca SUCCESS/OK durumundaki kaş kırpımlarını eğitim için kabul eder; dosyaların gerçekten var olup olmadığını kontrol eder.
4. Her frame'in hangi kaynak videodan geldiğini geri kazanır ve aynı videonun train/validation/test kümeleri arasında karışmadığını doğrular.
5. Kaş görüntülerini 224×224 boyutuna getirir, train aşamasında kontrollü augmentation uygular ve ImageNet/Swin V2 normalizasyonunu kullanır.
6. ImageNet üzerinde önceden eğitilmiş Swin V2 Tiny omurgasını alır; orijinal sınıflandırma kafasını kaldırıp real/fake için tek logit üreten yeni bir sınıflandırıcı ekler.
7. AdamW optimizasyonu, ayrı backbone/head learning rate'leri, BCEWithLogitsLoss, validation threshold seçimi, early stopping ve checkpoint sistemiyle eğitimi yürütür.
8. En iyi checkpoint'i taze bir model nesnesine yükler, test setini yalnız final değerlendirmede kullanır, frame-level ve mümkünse video-level tahminleri üretir.
9. Loss/accuracy/ROC-AUC grafikleri, confusion matrix, ROC ve Precision-Recall eğrileri ile JSON/CSV özetleri ve SHA256 manifesti üretir.

Eyebrow ROI Swin V2 Tiny - Kod Çalışma Mantığı ve Teknik Açıklama Raporu
Sayfa 3

Şekil 1. Notebook'un uçtan uca çalışma akışı.

3. Notebook'un Modüler Yapısı

Notebook çalışırken /content/eyebrow_swinv2_experiment altında geçici ve modüler bir Python paket yapısı oluşturur. Bu yaklaşım, tüm mantığı tek dev hücrede tutmak yerine veri, model, eğitim, değerlendirme ve yardımcı fonksiyonları ayrı modüllere ayırır.

Modül	Görev
deepfake_roi/data/dataset.py	Kaş ROI Dataset sınıfı, label dönüşümü, train/eval transformları ve ImageNet normalizasyonu.
deepfake_roi/models/swin.py	Swin V2 Tiny backbone ve binary classifier head tanımı.
deepfake_roi/training/engine.py	Epoch çalışma döngüsü, loss/olasılık toplama, metrikler, threshold seçimi, NaN/Inf kontrolleri.
deepfake_roi/evaluation/plots.py	Loss, accuracy, validation ROC-AUC, confusion matrix, ROC ve PR grafiklerinin üretimi.
deepfake_roi/utils/repro.py	Seed, RNG ve DataLoader worker reproducibility işlemleri.
deepfake_roi/utils/io.py	Atomik JSON/checkpoint kaydı, checkpoint doğrulama ve hash işlemleri.

Bu modüler yapı ayrıca çalıştırılan kaynak kodun snapshot'ının deney klasörüne kopyalanmasını sağlar. Böylece daha sonra notebook değişse bile ilgili run sırasında kullanılan Python kaynakları geriye dönük olarak incelenebilir.

4. Tek Doğruluk Kaynağı: YAML Konfigürasyonu

Notebook, dağınık biçimde hücrelere yazılmış hiperparametreler yerine tek bir YAML konfigürasyonunu "single source of truth" olarak kullanır. Eğitim kodu; veri yolu, görüntü boyutu, batch size, learning rate, augmentation, early stopping ve kalite kontrollerini bu yapıdan okur.

Kategori	Ayar	Değer / Anlam
Deney	region	eyebrow

Eyebrow ROI Swin V2 Tiny - Kod Çalışma Mantığı ve Teknik Açıklama Raporu

Kategori	Ayar	Değer / Anlam
Deney	model_name	swinv2_tiny
Deney	seed	42
Veri	image_size	224×224
Veri	batch_size	32
Veri	num_workers	2
Veri	split hedefi	train 0.80 / val 0.10 / test 0.10
Veri	split toleransı	±0.06, kaynak video düzeyinde sınıf bazlı
Model	pretrained	True - ImageNet1K_V1
Model	dropout	0.20
Eğitim	epochs	20
Eğitim	backbone_lr	1e-5
Eğitim	head_lr	1e-4
Eğitim	weight_decay	1e-2
Eğitim	gradient_clip_norm	1.0
Eğitim	mixed_precision	False
Eğitim	early_stopping_patience	6
Eğitim	monitor_metric	ROC-AUC; yoksa F1
Threshold	arama aralığı	0.05 - 0.95, 181 eşit adım
Threshold	seçim ölçütü	Validation F1 maksimumu
Kalite	smoke test	2 batch, FP32
Kalite	min. figür kısa kenar	600 px

5. local source package ve Dosya Yolu Mantığı

Kod local source package'ı Deney1_Source_Runtime altına mount eder. Ardından proje kökünü iki olası konumdan bulmaya çalışır: doğrudan LocalSource içindeki AISC DeepFake Çalışmaları klasörü veya paylaşılan local source kısayolunun .shortcut-targets-by-id yolu. İlk mevcut yol proje kökü olarak seçilir.

```
EYEBROW_ROI_ROOT = PROJECT_ROOT / "Deneyler/Nazlıcan/Deney 1/Kaş"
SELECTION_METADATA_PATH = PROJECT_ROOT / "Deneyler/Deney 1 Frame/secim_metadata.csv"
RESULTS_ROOT = PROJECT_ROOT / "Deneyler/Nazlıcan/Deney 1/Sonuçlar"
```

Bu tasarım, modeli farklı bir Colab hesabında çalıştırırken paylaşılan klasör/kısayol üzerinden aynı veri ve sonuç yapısını kullanabilmek için özellikle uygundur. Girdi klasörleri salt okunur mantıkla tüketilir; deney çıktıları yalnız Sonuçlar altındaki yeni run klasörüne yazılır.

6. Run ID, Çıktı Klasörü ve New/Resume Modu

Yeni deneyde run ID tarih-saat + bölge + model + seed formatında üretilir. Örnek: 20260807_1158_eyebrow_swinv2_tiny_seed42. Aynı klasör zaten varsa üzerine yazmak yerine hata verir; böylece önceki deney yanlışlıkla ezilmez.

Resume modunda ise yeni run oluşturulmaz. Kullanıcının verdiği resume_run_id ile mevcut run klasörü açılır ve last.ckpt üzerinden eğitim durumu geri yüklenir.

```
experiment:
  mode: resume
  resume_run_id: "20260807_1158_eyebrow_swinv2_tiny_seed42"
```

7. Reproducibility ve Ortam Kaydı

Kod deneyin yeniden üretilebilirliğini güçlendirmek için Python, NumPy ve PyTorch seed'lerini 42'ye sabitler. DataLoader için de ayrı bir torch.Generator ve worker_init_fn kullanılır. CUDA varsa GPU adı ve CUDA sürümü environment.json içine yazılır.

Ayrıca torch, torchvision, numpy, pandas, scikit-learn, matplotlib, Pillow, PyYAML ve tqdm sürümleri requirements_lock.txt dosyasına kaydedilir. Çalıştırılan kaynak modüllerin kopyası source_snapshot altında tutulur ve kaynak dosyaların hash'leri source_hash_manifest.json dosyasına yazılır.

8. Veri Kalitesi: Metadata Şema Kontrolü

Model eğitime başlamadan önce iki metadata tablosunun zorunlu sütunlara sahip olup olmadığını denetler. Kaş ROI metadata için sample_id, label, split, status, relative/absolute output path, input_path, frame_index, face_index, skip_reason ve run_id gibi alanlar beklenir. Frame seçim metadata tarafında ise sınıf, split, orijinal yol ve dosya adı alanları zorunludur.

Bu kontrolün amacı, upstream ROI çıkarma sürecinde dosya formatı değişmiş veya sütunlar eksilmişse modelin sessizce yanlış veriyle eğitilmesini önlemektir.

9. Veri Muhasebesi ve Kullanılabilir Örneklerin Seçilmesi

Kod ROI metadata içindeki status değerlerini büyük harfe normalize eder. Kabul edilen durumlar SUCCESS ve OK'dir. SKIPPED ve ERROR ayrı tutulur. Ardından şu eşitlik zorunlu olarak kontrol edilir:

```
total_inputs == success_count + skipped_count + error_count
```

Aynı zamanda sample_id alanlarının boş olmaması ve benzersiz olması gerekir. Bu adım, veri sayılarının raporlanan upstream pipeline ile tutarlı olduğunu kanıtlamaya yarar. Eğitime yalnız SUCCESS/OK satırları devam eder; SKIPPED görüntüler model girdisi olmaz.

10. Label ve Split Normalizasyonu

Label değerleri küçük harfe çevrilir ve yalnız real/fake kabul edilir. Split isimlerinde training→train,

validation/valid→val ve testing→test dönüşümleri yapılır. Her split'in hem real hem fake sınıfını içermesi zorunludur. Train, validation veya test'ten herhangi biri eksikse pipeline durur.

11. Kaş ROI Dosyalarının Gerçekten Var Olduğunun Kontrolü

Her metadata satırı için öncelikle output_relative_path kullanılarak Kaş klasörü altında tam dosya yolu oluşturulur. Relative path yoksa kullanılabilir bir absolute path denenir. SUCCESS/OK olduğu halde disk üzerinde bulunmayan ROI varsa model eğitimi başlamaz ve eksik satırlar missing_success_roi_files.csv dosyasına yazılır.

Bu davranış önemlidir çünkü yalnız metadata'da "başarılı" görünmesi yetmez; gerçek görüntü dosyasının da okunabilir olması gerekir.

12. Kaynak Video Kimliğinin Geri Kazanılması

Kaş metadata içindeki source_video bilgisine körü körüne güvenilmez. Kod frame adını input_path üzerinden çıkarır ve Deney 1 Frame/secim_metadata.csv tablosundaki dosya adıyla eşleştirir. Kaynak video adı, secim_metadata içindeki orijinal_yol alanının parent klasör adından deterministik biçimde geri kazanılır.

Eşleştirme many_to_one olarak doğrulanır. Frame seçim metadata dosyasında aynı dosya adı birden fazla kez geçerse veya kaş ROI satırı frame seçim tablosunda bulunamazsa pipeline durur. Ayrıca ROI metadata'daki label/split ile seçim metadata'daki label/split birebir karşılaştırılır.

```
metadata["source_video"] = metadata["source_video_recovered"]
metadata["video_id"] = metadata[label_col] + "__" + metadata["source_video"]
```

Label bilgisinin video_id içine eklenmesi, aynı isimli fakat farklı sınıf bağlamındaki kaynak video isimlerinin çakışmasını engellemeye yardımcı olur.

13. En Kritik Kontrol: Source-Video Leakage Önleme

Deepfake çalışmalarında aynı videodan alınmış farklı framelerin train ve test arasında dağılması model performansını yapay biçimde yükseltebilir. Bu nedenle notebook frame-level split'i olduğu

gibi kabul etmekle yetinmez; kaynak video kimliğini kullanarak video düzeyinde izolasyonu doğrular.

☐ Bir source video birden fazla label'a bağlanamaz.
☐ Bir source video birden fazla split'te bulunamaz.
☐ Train video kümesi validation ve test ile kesişemez.
☐ Validation ve test video kümeleri de kesişemez.
☐ Real ve fake sınıfları için source-video düzeyindeki train/val/test oranları yaklaşık 80/10/10 olmalı ve ±0.06 toleransı aşmamalıdır.

Bu kontrollerin tamamı split_leakage_report.json ile kayıt altına alınır. Bu bölüm notebook'un metodolojik olarak en önemli güvenlik katmanlarından biridir.

14. Görüntü Ön İşleme ve Augmentation

Her görüntü PIL ile açılır, RGB formatına çevrilir ve model girişinden önce 224×224 boyutuna yeniden ölçeklenir. Train ve evaluation transformları farklıdır.

Aşama	Train	Validation/Test
Resize	224×224	224×224
Horizontal flip	p=0.50	Yok
Rotation	±5°	Yok
Brightness	0.10	Yok
Contrast	0.10	Yok
Saturation	0.05	Yok
Hue	0.02	Yok
Tensor dönüşümü	Var	Var
Normalization	ImageNet/Swin V2	ImageNet/Swin V2

Augmentation yalnız train setine uygulanır. Validation ve test görüntülerinin rastgele dönüştürülmemesi, değerlendirme metriklerinin kararlı ve tekrar üretilebilir kalmasını sağlar. Kaş geometrisini fazla bozabilecek agresif dönüşümler kullanılmamıştır.

15. Normalizasyon Neden ImageNet Değerleriyle Yapılıyor?

Model ImageNet1K_V1 ağırlıklarıyla başlatıldığı için, önceden eğitilmiş Swin V2 Tiny'nin beklediği normalizasyon kullanılır: mean ≈ [0.485, 0.456, 0.406], std ≈ [0.229, 0.224, 0.225]. Proje train/val/test verisinden yeni mean/std öğrenilmez.

Bu karar iki amaca hizmet eder: Önceden eğitilmiş ağırlıkların beklediği giriş dağılımını korumak ve validation/test istatistiklerinin preprocessing'e sızmasını engellemek. normalization_audit.json içinde bu durum açıkça kaydedilir.

16. Dataset ve DataLoader Mantığı

EyebrowROIDataset her satır için görüntüyü yükler, transform uygular ve label'ı sayısal değere dönüştürür: real=0.0, fake=1.0. Ayrıca sample_id, video_id ve gerçek dosya yolunu da batch içinde taşır. Bu ek bilgiler daha sonra frame-level ve video-level tahmin dosyalarının üretilmesini sağlar.

Train DataLoader shuffle=True kullanır; validation ve test shuffle=False çalışır. Batch size 32'dir. CUDA kullanılıyorsa pin_memory etkinleştirilir. Train loader'ın generator state'i checkpoint içine kaydedildiği için resume sırasında veri sıralaması da mümkün olduğunca devam ettirilir.

17. Kullanılan Model: Swin V2 Tiny Binary Classifier

Model torchvision.models.swin_v2_t mimarisidir ve varsayılan olarak
Swin_V2_T_Weights.IMAGENET1K_V1 ağırlıklarıyla başlatılır. Bu nedenle model sıfırdan değil, genel görsel özellikleri daha önce öğrenmiş bir backbone üzerinden fine-tune edilir.
Orijinal Swin V2 Tiny sınıflandırma kafası kaldırılır ve nn.Identity ile değiştirilir. Backbone'un çıkardığı feature vektörü daha sonra özel binary head'e gönderilir:
LayerNorm(in_features)
Dropout(p=0.20)
Linear(in_features, 1)
Model forward aşamasında tek bir logit üretir. Sigmoid fonksiyonu model sınıfının içinde uygulanmaz; eğitim motoru metrik/olasılık hesabı yaparken logits üzerine sigmoid uygular. Bu, BCEWithLogitsLoss kullanımıyla numerik olarak doğru yaklaşımdır.
18. Loss Fonksiyonu ve Sınıf Ağırlığı
İkili sınıflandırma için BCEWithLogitsLoss kullanılır. Fake sınıfı pozitif sınıftır. Sınıf dengesizliği varsa
pos_weight yalnız train setindeki real/fake sayılarına göre hesaplanır:
pos_weight = num_real / num_fake
Validation veya test dağılımı loss ağırlığına karışmaz. Böylece değerlendirme verisi eğitim kararlarına doğrudan sızdırılmaz.
19. Optimizer, Learning Rate ve Scheduler
Optimizer AdamW'dir. Backbone ve yeni classifier head aynı optimizer içinde iki ayrı parametre grubu olarak tutulur. Önceden eğitilmiş backbone için daha küçük learning rate (1e-5), yeni eklenen classifier için daha büyük learning rate (1e-4) kullanılır. Bu, backbone bilgisini agresif biçimde bozmadan yeni göreve uyarlamayı hedefler.
Weight decay 1e-2'dir. Learning rate zamanla CosineAnnealingLR ile azaltılır. Scheduler T_max=20 ve minimum backbone learning rate'i başlangıcın %10'u olacak şekilde yapılandırılır.
20. Sayısal Sağlamlık Kontrolleri
Notebook daha tam eğitim başlamadan önce bir train batch'i üzerinde görüntü tensörlerinin shape, dtype, min, max, mean, std değerlerini kaydeder ve görüntü/label tensörlerinde NaN veya Inf olup olmadığını kontrol eder. Sonuç input_numerical_sanity.json dosyasına yazılır.
Ardından modelin kopyasıyla iki batch'lik FP32 smoke test çalıştırılır. Bu testte forward, loss, backward, gradient finiteness, gradient clipping ve optimizer step gerçekten yürütülür. Amaç, uzun eğitime başlamadan önce model-veri kombinasyonunun en az birkaç gerçek güncelleme adımını hatasız tamamlayabildiğini kanıtlamaktır.
21. Checkpoint Süreklilik Testi
Kod yalnız checkpoint yazmakla yetinmez; checkpoint'in gerçekten eksiksiz geri yüklenebildiğini de test eder. Validation batch'inde modelin logit ve loss çıktısı alınır, geçici checkpoint kaydedilir, taze

bir model/optimizer/scheduler/scaler nesnesi oluşturulup checkpoint yüklenir ve aynı batch yeniden çalıştırılır.
Logit ve loss değerlerinin belirlenen atol/rtol toleransları içinde aynı olması gerekir. Başarısız olursa

eğitim durur. Bu kontrol, Colab kesintisi sonrası resume yapılacağında checkpoint'in sadece ağırlıkları değil tüm gerekli durumu güvenilir biçimde taşıdığını doğrular.

22. Checkpoint İçinde Neler Saklanıyor?

Alan	Neden saklanıyor?
epoch	Hangi epoch'un tamamlandığını bilmek ve epoch+1'den devam etmek.
model_state_dict	Swin backbone + binary head ağırlıkları.
optimizer_state_dict	AdamW momentum/optimizer durumunu kaybetmemek.
scheduler_state_dict	Learning rate takvimine kaldığı yerden devam etmek.
scaler_state_dict	AMP kullanılırsa GradScaler durumu.
best_metric_score / best_threshold / best_epoch	En iyi validation durumunu korumak.
epochs_without_improvement	Early stopping sayacını sürdürmek.
history	Eğitim geçmişini korumak.
config	Resume sırasında deney kimliği ve ana ayarları doğrulamak.
rng_state	Python/NumPy/Torch/CUDA RNG durumunu geri yüklemek.
loader_generator_state	Train DataLoader sıralama durumunu sürdürmek.

23. True Resume Mantığı
Resume seçildiğinde last.ckpt yüklenir. Kod önce region, model_name, seed, image_size ve relative path column gibi deney kimliği alanlarının mevcut config ile uyuştuğunu kontrol eder. Uyuşmazlık varsa yanlış deney checkpoint'i ile devam edilmesine izin vermez.
Model, optimizer, scheduler, scaler, RNG, DataLoader generator, history, best metric, best threshold ve early stopping counter geri yüklenir. start_epoch = checkpoint_epoch + 1 olarak ayarlanır. Dolayısıyla resume doğru yapılandırıldığında eğitim mantıksal olarak kaldığı epoch'un sonrasından devam eder.
24. Eğitim Döngüsü Nasıl Çalışıyor?

Her epoch iki ana aşamadan oluşur: train ve validation. Train aşamasında optimizer aktif olduğu için forward + loss + backward + gradient clipping + optimizer step yapılır. Validation aşamasında torch.no_grad altında yalnız değerlendirme yapılır.
Train metrikleri varsayılan 0.50 threshold ile hesaplanır. Validation tarafında ise her epoch 0.05 ile 0.95 arasında 181 threshold denenir ve F1 skorunu en yüksek yapan threshold o epoch için seçilir. Model seçimi threshold'a bağlı F1 ile değil, öncelikle validation ROC-AUC ile yapılır. Eğer ROC-AUC hesaplanamazsa fallback olarak F1 kullanılır. Bu ayrım önemlidir: threshold operasyonel sınıflandırma noktasını belirlerken ROC-AUC modelin sıralama/ayrım gücünü threshold'dan bağımsız değerlendirir.

25. Early Stopping ve Best/Last Checkpoint

Validation monitor metriği iyileştiğinde best.ckpt güncellenir; her epoch sonunda last.ckpt her zaman yazılır. Ayrıca epoch_XXX.ckpt dosyası oluşturulur, ancak disk kullanımını sınırlamak için yalnız son 3 epoch checkpoint'i tutulur.
İyileşme 6 epoch boyunca gelmezse early stopping devreye girer. Checkpoint kaydı başarılı olduktan sonra training_history.csv güncellenir. Bu sıra, insan tarafından okunabilen history dosyasının checkpoint'ten daha ileride görünmesini önlemeyi amaçlar.

26. Final Test Neden "Fresh Best Checkpoint" ile Yapılıyor?

Eğitim bittikten sonra test doğrudan RAM'de duran modelle yapılmaz. Önce best.ckpt disktan okunur, pretrained=False ile taze bir model nesnesi oluşturulur ve checkpoint ağırlıkları bu modele yüklenir. Bir test batch'i ile forward yapılır ve logits'in finite olduğu doğrulanır.

Bu kontrol, "raporlanan test sonucu gerçekten diske kaydedilmiş best.ckpt modelinden üretildi" güvencesini sağlar. Ardından final threshold, best checkpoint içindeki validation-selected threshold'dan alınır. Test seti threshold seçmek veya model seçmek için kullanılmaz; yalnız final evaluation için kullanılır.

27. Frame-Level Tahminler

Her test frame için model fake_probability üretir. Bu olasılık final threshold ile karşılaştırılarak predicted_label hesaplanır. test_frame_predictions.csv içinde sample_id, video_id, image_path, true_label, fake_probability, predicted_label, threshold, true_class ve predicted_class alanları saklanır.
Frame-level metrikler accuracy, precision, recall, F1, ROC-AUC ve PR-AUC olarak hesaplanır ve test_frame_metrics.json dosyasına yazılır.

28. Video-Level Tahminler

Kaynak video kimliği güvenilir biçimde kurtarıldığı için kod frame tahminlerini video_id bazında gruplar. Aynı videoya ait frame fake_probability değerlerinin aritmetik ortalaması alınır. Bu ortalama final threshold ile karşılaştırılarak video sınıfı belirlenir.
video_fake_probability = mean(frame_fake_probabilities)
predicted_video_label = video_fake_probability >= FINAL_THRESHOLD
Bu yöntem temporal bir video modeli değildir; frame'leri bağımsız işler ve yalnız test sonrası olasılık agregasyonu yapar. Bunun avantajı basitlik, sınırlılığı ise zaman içindeki hareket/dinamik bilgiyi doğrudan modellememesidir.

29. Üretilen Grafikler

Dosya	Ne gösterir?
training_validation_loss.png	Epoch bazında train ve validation loss.
training_validation_accuracy.png	Train ve validation accuracy değişimi.
validation_roc_auc.png	Validation ROC-AUC'nin epochlara göre seyri.
test_confusion_matrix.png	Real/Fake için TN, FP, FN, TP dağılımı.
test_roc_curve.png	Test ROC eğrisi ve ROC-AUC.
test_precision_recall_curve.png	Precision-Recall eğrisi ve Average Precision.

Grafikler İngilizce etiketlerle oluşturulur ve kaydedildikten sonra kısa kenarın en az 600 piksel olduğu kontrol edilir. Kalite şartını sağlamayan görsel final kalite kapısından geçemez.

30. Final Çıktı Yapısı

```
<run_id>/
├── checkpoints/
│   ├── best.ckpt
│   ├── last.ckpt
│   └── epoch_XXX.ckpt
├── logs/pipeline.log
├── metrics/
│   ├── training_history.csv
│   ├── test_frame_metrics.json
│   └── test_video_metrics.json
├── predictions/
│   ├── test_frame_predictions.csv
│   └── test_video_predictions.csv
├── figures/
│   ├── training_validation_loss.png
```

```
│   ├── training_validation_accuracy.png
│   ├── validation_roc_auc.png
│   ├── test_confusion_matrix.png
│   ├── test_roc_curve.png
│   └── test_precision_recall_curve.png
├── artifacts/
│   ├── source_snapshot/
│   ├── metadata_schema_audit.json
│   ├── data_accounting.json
│   ├── split_leakage_report.json
│   ├── metadata_used.csv
│   ├── dataset_summary.csv
│   ├── normalization_audit.json
│   ├── input_numerical_sanity.json
│   ├── checkpoint_continuity_test.json
│   └── source_hash_manifest.json
├── config_resolved.yaml
├── requirements_lock.txt
├── environment.json
├── run_summary.json
└── output_manifest.csv
```

31. Output Manifest ve SHA256 Neden Var?

En son aşamada run klasöründeki tüm dosyalar taranır ve relative_path, category, size_bytes ve sha256 alanlarıyla output_manifest.csv oluşturulur. Böylece deney çıktılarından herhangi birinin daha sonra değiştirilip değiştirilmediği veya farklı bir dosyayla karışıp karışmadığı hash karşılaştırmasıyla denetlenebilir.

Manifest oluşturulmadan hemen önce final log satırları flush edilir; bundan sonra run dosyalarının değiştirilmemesi hedeflenir. Bu yaklaşım deney izlenebilirliğini ve arşiv bütünlüğünü artırır.

32. Final Quality Gates

Notebook bir deney klasörünü “tamamlandı” kabul etmeden önce gerekli tüm ana dosyaların varlığını kontrol eder. Son anda leakage kontrolünü tekrarlar, figür çözünürlüklerini tekrar doğrular ve best.ckpt’in yeniden yüklenebildiğini test eder. Tüm kapılar geçerse pipeline.log içine ALL FINAL QUALITY GATES PASSED yazar.

Kalite Kapısı	Kontrol ettiği risk
Metadata schema	Eksik/yanlış sütunlarla sessiz eğitim.
Accounting equality	Veri sayılarının upstream pipeline ile tutarsızlığı.
Source-video split isolation	Aynı videonun train/val/test arasında sızması.
Input numerical sanity	Bozuk/NaN/Inf tensörler.
2-batch FP32 smoke test	Uzun eğitimden önce forward/backward kararsızlığı.
Checkpoint continuity	Checkpoint’in tam ve doğru geri yüklenememesi.
Fresh best checkpoint inference	Raporlanan testin gerçekten kayıtlı best modelden gelmemesi.
Required outputs	Eksik metrik/prediction/figür/artifact.
Figure resolution	Düşük çözünürlüklü rapor grafikleri.
Output manifest	Dosya bütünlüğünün sonradan doğrulanamaması.

33. Kodun Güçlü Yönleri

☐ Kaş ROI’ye odaklı net ve izole bir baseline deney tanımlar.
☐ ImageNet ön-eğitimli Swin V2 Tiny ile transfer learning kullanır.
☐ Train/validation/test leakage’i frame seviyesinde değil kaynak video seviyesinde kontrol eder.
☐ Test verisini model/threshold seçimi için kullanmaz; yalnız final değerlendirmede kullanır.
☐ Checkpoint yalnız model ağırlıklarını değil optimizer, scheduler, RNG ve loader state’i de içerir.

☐ Resume öncesi deney kimliği kontrolü yapar.
☐ Kaynak kod snapshot/hashing, environment lock ve output manifest ile yüksek izlenebilirlik sağlar.
☐ NaN/Inf, smoke test ve checkpoint continuity gibi pratik kalite kontrolleri içerir.

34. Kodun Sınırları ve Dikkat Edilmesi Gerekenler

☐ Model yalnız kaş ROI görüntüsünü görür; yüzün diğer bölgelerindeki deepfake artefaktlarını kullanamaz.
☐ Video-level sonuç gerçek temporal modelleme değildir; frame olasılıklarının ortalamasıdır.
☐ 224×224’e doğrudan resize, çok dar ROI’lerde en-boy oranını değiştirebilir; bu metodolojik tercih ileride test edilebilir.
☐ Kaynak video geri kazanımı frame dosya adlarının seçim metadata’sında benzersiz olmasına bağlıdır.
☐ Threshold validation F1’e göre seçilir. Uygulamada false-positive/false-negative maliyetleri farklıysa başka threshold stratejileri gerekebilir.
☐ Swin V2 Tiny pretrained olduğu için bu deney “sıfırdan CNN eğitimi” değildir; sonuçlar transfer

learning baseline olarak raporlanmalıdır.
☐ Kaynak video izolasyonu kontrol edilse de kişi/kimlik bazlı ayrım ayrıca garanti edilmez; aynı kişinin farklı videoları farklı splitlerdeyse kimlik bazlı genelleme ayrı bir deney konusu olabilir.
35. Bir Frame Baştan Sona Nasıl İşleniyor?
1. metadata.csv içindeki SUCCESS/OK satırı seçilir.
2. output_relative_path ile Kaş klasöründeki gerçek ROI dosyası bulunur.
3. input_path içinden frame adı çıkarılır ve secim_metadata.csv ile eşleştirilerek source_video geri kazanılır.
4. Split ve label tutarlılığı kontrol edilir; source-video leakage yoksa satır dataset'e girer.
5. Görüntü RGB'ye çevrilir, 224×224 resize edilir; train ise augmentation uygulanır; ImageNet normalizasyonu yapılır.
6. Tensor Swin V2 Tiny backbone'dan geçirilir ve classifier tek logit üretir.
7. Train ise BCEWithLogitsLoss hesaplanır, backward ve AdamW update yapılır; validation/test ise yalnız olasılık üretilir.
8. Logit sigmoid ile fake_probability değerine çevrilir.
9. Validation sırasında threshold seçilir; final testte best checkpoint'in validation threshold'u kullanılır.
10. Frame tahmini CSV'ye, ilgili video tahmini ise frame olasılıklarının ortalamasıyla video CSV'sine yazılır.
36. Hücre Bazlı Notebook Haritası
Hücre İçerik
0 Amaç, veri kaynağı ve kalite ilkeleri açıklaması.
1 Opsiyonel pinned yardımcı paketler.
2 local source package mount.
3-9 Modüler runtime paketinin oluşturulması:
utils/data/model/training/plots.
10 Modüler paket importları.
11 YAML single-source-of-truth config.
12 local source path çözümleme, run klasörü, log ve new/resume
modu.

Hücre İçerik
13 Environment lock, source snapshot ve hash.
14 Seed ve device seçimi.
15 Metadata, accounting, source-video recovery ve leakage
gates.
16 Normalization, dataset ve dataloader.
17 Model, class-weighted loss, AdamW ve scheduler.
18 Input sanity ve 2-batch FP32 smoke test.
19 Checkpoint state ve continuity test.
20 True resume logic.
21 Train/validation eğitim döngüsü, threshold, early stopping,
checkpointler.
22 Fresh best.ckpt inference ve final frame test.
23 Frame/video predictions ve video metrics.
24 Grafik üretimi.
25 Final quality gates, run summary ve manifest.
26 Resume kullanım notları.
37. Sonuç
Bu notebook, yalnızca "Swin V2 Tiny modeli eğiten" basit bir eğitim dosyası değildir. Asıl yapısı, kaş ROI üzerinden real/fake sınıflandırma deneyini tekrarlanabilir, denetlenebilir ve veri sızıntısına karşı kontrollü hale getiren uçtan uca bir deney pipeline'ıdır.
Model tarafında ImageNet ön-eğitimli Swin V2 Tiny kullanılır; kaş görüntülerinden learned feature'lar otomatik çıkarılır ve binary classifier ile real/fake kararı verilir. Eğitim tarafında ayrı backbone/head learning rate'leri, BCEWithLogitsLoss, AdamW, cosine scheduler, validation tabanlı threshold, ROC-AUC ile model seçimi ve early stopping uygulanır.
Metodolojik olarak en önemli noktalar ise source-video leakage kontrolü, test setinin yalnız final aşamada kullanılması, checkpoint continuity ve fresh-best-checkpoint inference kapılarıdır. Bu nedenle ortaya çıkan sonuç yalnız bir skor değil; hangi veriyle, hangi kodla ve hangi checkpoint ile üretildiği geriye dönük olarak izlenebilen bir deney çıktısıdır.